

Prediction Market Bot: Week-1 Improvement Plan

⚠ Critical Finding Before We Start

Manifold Markets sunset real-money trading on March 28, 2025. It now operates entirely on play-money (Mana). This means cross-platform arbitrage between Polymarket and Manifold is **not viable for real profit**. Your arbitrage strategy must focus on **Polymarket vs Kalshi** (the two real-money platforms). Manifold remains useful as a free signal source and backtesting sandbox.

Executive Summary: Top 3 Quick Wins

Rank	Improvement	Expected Impact	Implementation Time	Risk Reduction
1	Circuit breakers + trailing stop-loss	Prevents catastrophic losses like the -\$47/- \$43 disasters	2-3 hours	★★★★★
2	Polymarket intra-event arbitrage scanner	+\$0.50–\$2.00/day risk-free profit on mispriced multi-outcome events	4-6 hours	★★★★☆
3	Liquidity + concentration filters	Avoids illiquid traps, caps domain exposure at 20%	1-2 hours	★★★★☆

Bottom line: Your two -\$40+ losses account for more than 5x your total profit. Fixing risk management first will have a bigger impact than finding new alpha.

Implementation Roadmap

Day 1 (Monday): Risk Management — Stop the Bleeding

Morning: Circuit Breakers + Hard Limits

```
python
```

risk_manager.py — Add to your existing bot

```
import sqlite3
```

```
from datetime import datetime, timedelta
```

```
class RiskManager:
```

```
    def __init__(self, db_path="trades.db"):
```

```
        self.db_path = db_path
```

```
        # --- CONFIGURABLE LIMITS ---
```

```
        self.MAX_DAILY_LOSS = 50.00    # Stop trading if down $50 today
```

```
        self.MAX_POSITION_PCT = 0.20   # Max 20% of capital in one domain
```

```
        self.MAX_CORRELATED = 3        # Max 3 positions in same domain
```

```
        self.MAX_SINGLE_POSITION = 10.00 # Max $10 per position
```

```
        self.COOLDOWN_MINUTES = 60     # Pause after circuit breaker trips
```

```
        self._circuit_broken = False
```

```
        self._cooldown_until = None
```

```
    def get_daily_pnl(self) -> float:
```

```
        """Sum of realized + unrealized P&L since midnight UTC."""
```

```
        conn = sqlite3.connect(self.db_path)
```

```
        today = datetime.utcnow().strftime("%Y-%m-%d")
```

```
        cursor = conn.execute("""
```

```
            SELECT COALESCE(SUM(pnl), 0) FROM trades
```

```
            WHERE date(closed_at) = ? AND status = 'closed'
```

```
        """, (today,))
```

```
        realized = cursor.fetchone()[0]
```

```
        cursor = conn.execute("""
```

```
            SELECT COALESCE(SUM(current_value - entry_cost), 0)
```

```
            FROM positions WHERE status = 'open'
```

```
        """)
```

```
        unrealized = cursor.fetchone()[0]
```

```
        conn.close()
```

```
        return realized + unrealized
```

```
    def check_circuit_breaker(self) -> bool:
```

```
        """Returns True if trading should STOP."""
```

```
        # Check cooldown
```

```
        if self._cooldown_until and datetime.utcnow() < self._cooldown_until:
```

```
            return True
```

```
        daily_pnl = self.get_daily_pnl()
```

```

if daily_pnl <= -self.MAX_DAILY_LOSS:
    self._circuit_broken = True
    self._cooldown_until = datetime.utcnow() + timedelta(
        minutes=self.COOLDOWN_MINUTES
    )
    self._alert(f"🚨 CIRCUIT BREAKER: Daily P&L = ${daily_pnl:.2f}")
    return True
return False

```

```

def check_concentration(self, domain: str, capital: float) -> bool:
    """Returns True if we can add a position in this domain."""
    conn = sqlite3.connect(self.db_path)
    cursor = conn.execute("""
        SELECT COUNT(*), COALESCE(SUM(entry_cost), 0)
        FROM positions
        WHERE status = 'open' AND domain = ?
    """, (domain,))
    count, exposure = cursor.fetchone()
    conn.close()

    if count >= self.MAX_CORRELATED:
        return False
    if (exposure / max(capital, 1)) >= self.MAX_POSITION_PCT:
        return False
    return True

```

```

def check_position_size(self, amount: float) -> float:
    """Clamp position size to maximum allowed."""
    return min(amount, self.MAX_SINGLE_POSITION)

```

```

def _alert(self, message: str):
    """Send alert — hook into n8n webhook."""
    import requests
    webhook_url = "YOUR_N8N_WEBHOOK_URL"
    try:
        requests.post(webhook_url, json={
            "text": message,
            "timestamp": datetime.utcnow().isoformat()
        }, timeout=5)
    except Exception:
        pass # Don't crash the bot over alerts

```

Afternoon: Trailing Stop-Loss

python

exit_manager.py — Replace your fixed stop-loss logic

class TrailingStopManager:

"""

Replaces fixed -15%/-25% stops with trailing stops that
lock in profits as positions move in your favor.

"""

def __init__(self):

Tiered trailing config

self.TIERS = [

(profit_threshold, trail_pct, description)

(0.00, 0.15, "initial: 15% trailing stop"),

(0.05, 0.10, "in profit 5%+: tighten to 10%"),

(0.15, 0.07, "in profit 15%+: tighten to 7%"),

(0.25, 0.05, "in profit 25%+: lock 5% trail"),

]

def calculate_stop_price(self, entry_price, current_price,
high_water_mark):

"""

Returns the stop-loss price based on trailing logic.

entry_price: what you paid

current_price: where it is now

high_water_mark: highest price seen since entry

"""

hwm = max(high_water_mark, current_price)

profit_pct = (hwm - entry_price) / entry_price if entry_price else 0

Find which tier applies

trail_pct = self.TIERS[0][1] # default

for threshold, pct, _ in self.TIERS:

if profit_pct >= threshold:

trail_pct = pct

stop_price = hwm * (1 - trail_pct)

return max(stop_price, entry_price * 0.75) # Hard floor: -25%

def should_exit(self, entry_price, current_price, high_water_mark):

stop = self.calculate_stop_price(

entry_price, current_price, high_water_mark

)

```
return current_price <= stop, stop
```

```
# --- PARTIAL EXIT / SCALE-OUT LOGIC ---
```

```
class ScaleOutManager:
```

```
    """
```

```
    Scale out of positions at multiple profit levels  
    instead of all-or-nothing at +10%/+20%.
```

```
    """
```

```
    SCALE_LEVELS = [
```

```
        # (profit_pct, sell_fraction, description)
```

```
        (0.05, 0.25, "sell 25% at +5%"),
```

```
        (0.15, 0.35, "sell 35% at +15%"),
```

```
        (0.25, 0.40, "sell remaining 40% at +25%"),
```

```
    ]
```

```
def get_exit_actions(self, entry_price, current_price,  
                    remaining_fraction):
```

```
    """
```

```
    Returns (fraction_to_sell, level_name) or (0, None).  
    Track remaining_fraction in your positions table.
```

```
    """
```

```
    if remaining_fraction <= 0:
```

```
        return 0, None
```

```
    profit_pct = (current_price - entry_price) / entry_price
```

```
    for level_pct, sell_frac, desc in self.SCALE_LEVELS:
```

```
        if profit_pct >= level_pct:
```

```
            actual_sell = min(sell_frac, remaining_fraction)
```

```
            return actual_sell, desc
```

```
    return 0, None
```

Database schema addition for trailing stops:

```
sql
```

```
-- Run this against your existing SQLite database
ALTER TABLE positions ADD COLUMN high_water_mark REAL DEFAULT 0;
ALTER TABLE positions ADD COLUMN remaining_fraction REAL DEFAULT 1.0;
ALTER TABLE positions ADD COLUMN domain TEXT DEFAULT 'unknown';
ALTER TABLE positions ADD COLUMN last_checked_at TEXT;

-- Index for fast domain queries
CREATE INDEX IF NOT EXISTS idx_positions_domain
  ON positions(domain, status);

-- Daily P&L tracking table
CREATE TABLE IF NOT EXISTS daily_snapshots (
  date TEXT PRIMARY KEY,
  realized_pnl REAL DEFAULT 0,
  unrealized_pnl REAL DEFAULT 0,
  open_positions INTEGER DEFAULT 0,
  circuit_breaker_triggered INTEGER DEFAULT 0
);
```

Day 2-3 (Tuesday–Wednesday): Arbitrage Detection

Focus: Polymarket Intra-Event Arbitrage

The most reliable, lowest-risk arbitrage in prediction markets isn't cross-platform — it's within Polymarket's own multi-outcome events. When an event has N mutually exclusive outcomes, the sum of YES prices across all outcomes should equal \$1.00. When it doesn't, there's free money.

```
python
```

```
# arbitrage_scanner.py
```

```
import time
```

```
import json
```

```
import requests
```

```
from dataclasses import dataclass
```

```
from typing import Optional
```

```
GAMMA_API = "https://gamma-api.polymarket.com"
```

```
@dataclass
```

```
class ArbitrageOpportunity:
```

```
    event_title: str
```

```
    event_slug: str
```

```
    opportunity_type: str # "overpriced" or "underpriced"
```

```
    total_yes_cost: float # sum of all YES prices
```

```
    profit_cents: float # cents profit per $1 risked
```

```
    markets: list # individual market details
```

```
    timestamp: str
```

```
class PolymarketArbitrageScanner:
```

```
    """
```

```
    Scans Polymarket for intra-event arbitrage.
```

```
    Strategy 1 — "Overpriced event": Sum of YES prices > $1.00
```

```
    → Buy NO on every outcome. Guaranteed one NO pays $1.
```

```
    → Cost = N * $1 - sum(YES). If sum(YES) > $1 → profit.
```

```
    → Actually: buy NO on each = $1-YES_price each.
```

```
    Total NO cost = N - sum(YES). Payout = N-1.
```

```
    Profit when sum(YES) > 1.
```

```
    Strategy 2 — "Underpriced event": Sum of YES prices < $1.00
```

```
    → Buy YES on every outcome. One will pay $1.
```

```
    → Cost = sum(YES). Payout = $1. Profit = 1 - sum.
```

```
    """
```

```
    def __init__(self, min_profit_cents=2.0, min_volume=5000):
```

```
        self.min_profit_cents = min_profit_cents
```

```
        self.min_volume = min_volume
```

```
    def fetch_active_events(self) -> list:
```

```
        """Get multi-outcome events from Gamma API."""
```

```
        events = []
```



```
offset = 0
```

```
while True:
```

```
    resp = requests.get(
```

```
        f'{GAMMA_API}/events',
```

```
        params={
```

```
            "active": True,
```

```
            "closed": False,
```

```
            "limit": 100,
```

```
            "offset": offset
```

```
        },
```

```
        timeout=10
```

```
    )
```

```
    batch = resp.json()
```

```
    if not batch:
```

```
        break
```

```
    events.extend(batch)
```

```
    offset += 100
```

```
    if len(batch) < 100:
```

```
        break
```

```
    time.sleep(0.3) # respect rate limits
```

```
return events
```

```
def analyze_event(self, event: dict) -> Optional[ArbitrageOpportunity]:
```

```
    """Check if a multi-outcome event has arbitrage."""
```

```
    markets = event.get("markets", [])
```

```
    if len(markets) < 2:
```

```
        return None
```

```
    # Build price data
```

```
    market_data = []
```

```
    total_yes = 0.0
```

```
    total_volume = 0.0
```

```
    for m in markets:
```

```
        # outcomePrices is a JSON string: "["0.55", "0.45"]"
```

```
        prices_str = m.get("outcomePrices", "")
```

```
        try:
```

```
            prices = json.loads(prices_str)
```

```
            yes_price = float(prices[0]) if prices else 0
```

```
        except (json.JSONDecodeError, IndexError, ValueError):
```

```
            return None
```

```
    volume = float(m.get("volume", 0) or 0)
```

```
    total_volume += volume
```

```

total_yes += yes_price

market_data.append({
    "question": m.get("question", ""),
    "yes_price": yes_price,
    "no_price": 1 - yes_price,
    "volume": volume,
    "condition_id": m.get("conditionId", ""),
    "clob_token_ids": m.get("clobTokenIds", ""),
})

if total_volume < self.min_volume:
    return None

# Check for arbitrage
opp_type = None
profit = 0.0

if total_yes < 1.0:
    # Underpriced: buy all YES, one wins $1
    profit = (1.0 - total_yes) * 100 # in cents
    opp_type = "underpriced"
elif total_yes > 1.0:
    # Overpriced: buy all NO
    total_no_cost = len(markets) - total_yes
    # Payout: (N-1) * $1 (since exactly one outcome wins,
    # all other NOs pay $1)
    payout = len(markets) - 1
    profit = (payout - total_no_cost) * 100
    opp_type = "overpriced"

if profit < self.min_profit_cents:
    return None

return ArbitrageOpportunity(
    event_title=event.get("title", ""),
    event_slug=event.get("slug", ""),
    opportunity_type=opp_type,
    total_yes_cost=total_yes,
    profit_cents=round(profit, 2),
    markets=market_data,
    timestamp=time.strftime("%Y-%m-%dT%H:%M:%SZ")
)

```

```

def scan(self) -> list:
    """Full scan. Returns list of ArbitrageOpportunity."""
    events = self.fetch_active_events()
    opportunities = []
    for event in events:
        opp = self.analyze_event(event)
        if opp:
            opportunities.append(opp)
    # Sort by profit descending
    opportunities.sort(key=lambda x: x.profit_cents, reverse=True)
    return opportunities

```

--- CROSS-PLATFORM SCANNER (Polymarket vs Kalshi) ---

```

class CrossPlatformScanner:

```

```

    """

```

Compares prices on the same event across Polymarket and Kalshi.

The challenge: Markets are named differently on each platform, so we need fuzzy matching. This uses a manual mapping approach for reliability (fuzzy matching has too many false positives).

```

    """

```

Manually curated mappings — update weekly

```

MARKET_MAPPINGS = [
    {
        "name": "BTC hourly up/down",
        "poly_slug": "bitcoin-up-or-down",
        "kalshi_series": "KXBTC",
        "match_type": "hourly_btc"
    },
    # Add more as you discover them
]

```

```

def compare_btc_hourly(self) -> list:

```

```

    """

```

BTC hourly markets are the most liquid cross-platform arb. Both platforms offer hourly Bitcoin price direction markets.

```

    """

```

```

    opportunities = []

```

Fetch from Polymarket

```

    poly_resp = requests.get(

```

```
f"{GAMMA_API}/events",
params={"slug": "bitcoin-up-or-down", "active": True},
timeout=10
)
```

Fetch from Kalshi

```
kalshi_resp = requests.get(
    "https://api.elections.kalshi.com/v1/events",
    params={"series_ticker": "KXBTC", "status": "open"},
    timeout=10
)
```

Compare YES prices on matching strike/expiry pairs

(Implementation depends on current market structure)

Key insight: if Poly YES + Kalshi NO < \$1.00, profit exists

return opportunities

--- RESOLUTION ARBITRAGE ---

class ResolutionArbitrageScanner:

"""

Find markets closing in <24h where the outcome is near-certain
but the price hasn't converged to \$0.99+.

These are low-risk: the event resolves soon, and the outcome
is almost known. Buy at \$0.95, collect \$1.00 in hours.

"""

def scan_near_resolution(self, min_price=0.90, max_price=0.98):

"""

Find markets where:

- Closes within 24 hours
- YES or NO price is between min_price and max_price
- Volume > \$1,000 (liquid enough to exit)

"""

```
resp = requests.get(
    f"{GAMMA_API}/markets",
    params={
        "active": True,
        "closed": False,
        "limit": 200,
        "order": "endDate",
```

```

        "ascending": True, # soonest-closing first
    },
    timeout=10
)
markets = resp.json()
candidates = []

now = time.time()
cutoff = now + 86400 # 24 hours

for m in markets:
    end_date = m.get("endDate", "")
    if not end_date:
        continue

    # Parse ISO date
    try:
        from datetime import datetime
        end_dt = datetime.fromisoformat(
            end_date.replace("Z", "+00:00")
        )
        end_ts = end_dt.timestamp()
    except (ValueError, TypeError):
        continue

    if end_ts > cutoff:
        break # Past 24h window, stop

    prices = json.loads(m.get("outcomePrices", "[]"))
    if not prices:
        continue

    yes_price = float(prices[0])
    volume = float(m.get("volume", 0) or 0)

    if volume < 1000:
        continue

    # Check if near-certain outcome
    if min_price <= yes_price <= max_price:
        candidates.append({
            "question": m.get("question"),
            "yes_price": yes_price,
            "profit_per_share": round((1 - yes_price) * 100, 1),

```

```
        "hours_to_resolution": round(
            (end_ts - now) / 3600, 1
        ),
        "volume": volume,
        "slug": m.get("slug", ""),
    })
elif (1 - yes_price) >= min_price and \
    (1 - yes_price) <= max_price:
    candidates.append({
        "question": m.get("question"),
        "no_price": 1 - yes_price,
        "profit_per_share": round(yes_price * 100, 1),
        "hours_to_resolution": round(
            (end_ts - now) / 3600, 1
        ),
        "volume": volume,
        "slug": m.get("slug", ""),
    })

return candidates
```

n8n Webhook Integration for Arbitrage Alerts:

python

```
# alert_dispatcher.py — Send arb alerts to your n8n workflow
```

```
import requests
```

```
import json
```

```
def send_arb_alert(opportunity, webhook_url):
```

```
    """
```

```
    Push arbitrage opportunity to n8n webhook.
```

```
    In n8n, create a Webhook node → filter → Slack/Telegram/Email.
```

```
    """
```

```
    payload = {
```

```
        "type": "arbitrage_alert",
```

```
        "event": opportunity.event_title,
```

```
        "profit_cents": opportunity.profit_cents,
```

```
        "opportunity_type": opportunity.opportunity_type,
```

```
        "total_yes_cost": opportunity.total_yes_cost,
```

```
        "market_count": len(opportunity.markets),
```

```
        "markets": [
```

```
            {
```

```
                "question": m["question"],
```

```
                "yes_price": m["yes_price"]
```

```
            }
```

```
            for m in opportunity.markets
```

```
        ],
```

```
        "timestamp": opportunity.timestamp,
```

```
        "action_url": f"https://polymarket.com/event/{opportunity.event_slug}"
```

```
    }
```

```
    requests.post(webhook_url, json=payload, timeout=5)
```

```
def run_scanner_loop(webhook_url, interval_seconds=300):
```

```
    """Run arbitrage scanner every 5 minutes."""
```

```
    scanner = PolymarketArbitrageScanner(
```

```
        min_profit_cents=2.0,
```

```
        min_volume=5000
```

```
    )
```

```
    while True:
```

```
        try:
```

```
            opps = scanner.scan()
```

```
            for opp in opps:
```

```
                send_arb_alert(opp, webhook_url)
```

```
            print(
```

```
                f"[ARB] {opp.event_title}: "
```

```
        f"+{opp.profit_cents}¢ ({opp.opportunity_type})"
    )
except Exception as e:
    print(f"[ERROR] Scanner: {e}")
time.sleep(interval_seconds)
```

Day 4-5 (Thursday–Friday): Entry Signal Enhancements

```
python
```



```
# signal_filters.py — Add these filters BEFORE your entry logic
```

```
import requests
import json
```

```
class LiquidityFilter:
```

```
    """
```

```
    Skip markets that are too illiquid to trade safely.
```

```
    Illiquid markets = wide spreads = guaranteed losses.
```

```
    """
```

```
    def __init__(self, min_volume=5000, min_traders=100,
                  max_spread=0.10):
```

```
        self.min_volume = min_volume
```

```
        self.min_traders = min_traders
```

```
        self.max_spread = max_spread
```

```
    def passes(self, market: dict) -> bool:
```

```
        volume = float(market.get("volume", 0) or 0)
```

```
        if volume < self.min_volume:
```

```
            return False
```

```
        # Check spread from order book
```

```
        clob_token_ids = market.get("clobTokenIds", "")
```

```
        if clob_token_ids:
```

```
            try:
```

```
                tokens = json.loads(clob_token_ids)
```

```
                yes_token = tokens[0]
```

```
                book = self._get_order_book(yes_token)
```

```
                spread = self._calculate_spread(book)
```

```
                if spread > self.max_spread:
```

```
                    return False
```

```
            except (json.JSONDecodeError, IndexError):
```

```
                pass
```

```
        return True
```

```
    def _get_order_book(self, token_id: str) -> dict:
```

```
        from py_clob_client.client import ClobClient
```

```
        client = ClobClient("https://clob.polymarket.com")
```

```
        return client.get_order_book(token_id)
```

```
def _calculate_spread(self, book: dict) -> float:
    bids = book.get("bids", [])
    asks = book.get("asks", [])
    if not bids or not asks:
        return 1.0 # No liquidity = max spread
    best_bid = float(bids[0].get("price", 0))
    best_ask = float(asks[0].get("price", 1))
    return best_ask - best_bid
```

```
class MetaculusComparison:
```

```
    """
```

Compare Polymarket prices against Metaculus community forecasts.
If Polymarket says 60% but Metaculus superforecasters say 80%,
that's a potential mispricing signal.

Free API: <https://www.metaculus.com/api/>

```
    """
```

```
METACULUS_API = "https://www.metaculus.com/api2"
```

```
def search_questions(self, query: str, limit=5) -> list:
```

```
    """Search Metaculus for matching questions."""
```

```
    resp = requests.get(
        f"{self.METACULUS_API}/questions/",
        params={
            "search": query,
            "limit": limit,
            "status": "open",
            "type": "forecast",
            "order_by": "-activity",
        },
        timeout=10
    )
```

```
    if resp.status_code != 200:
        return []
```

```
    results = resp.json().get("results", [])
```

```
    return [
        {
            "title": q.get("title"),
            "community_prediction": q.get(
                "community_prediction", {}
            ).get("full", {}).get("q2"),
```

```

        "url": f"https://metaculus.com {q.get('page_url', '')}",
        "num_forecasters": q.get("number_of_forecasters", 0),
    }
    for q in results
    if q.get("community_prediction")
]

```

```

def find_mispricing(self, poly_question: str,
                    poly_price: float,
                    threshold=0.10) -> dict:
    """
    Returns mispricing info if Metaculus diverges from
    Polymarket by more than threshold (default 10%).
    """
    matches = self.search_questions(poly_question, limit=3)
    if not matches:
        return None

    best_match = matches[0]
    meta_prob = best_match["community_prediction"]
    if meta_prob is None:
        return None

    divergence = meta_prob - poly_price
    if abs(divergence) >= threshold:
        return {
            "polymarket_price": poly_price,
            "metaculus_probability": round(meta_prob, 3),
            "divergence": round(divergence, 3),
            "direction": "BUY YES" if divergence > 0 else "BUY NO",
            "metaculus_question": best_match["title"],
            "metaculus_url": best_match["url"],
            "num_forecasters": best_match["num_forecasters"],
        }
    return None

```

```

class OrderBookImbalanceDetector:

```

```

    """
    Detect one-sided order books that signal momentum.
    If there's 10x more buy pressure than sell pressure,
    the price is likely to move up.
    """

```

```

def analyze_imbalance(self, token_id: str) -> dict:
    from py_clob_client.client import ClobClient
    client = ClobClient("https://clob.polymarket.com")
    book = client.get_order_book(token_id)

    bid_volume = sum(
        float(b.get("size", 0)) for b in book.get("bids", [])
    )
    ask_volume = sum(
        float(a.get("size", 0)) for a in book.get("asks", [])
    )

    total = bid_volume + ask_volume
    if total == 0:
        return {"imbalance": 0, "signal": "no_data"}

    # Imbalance ratio: positive = buy pressure, negative = sell
    imbalance = (bid_volume - ask_volume) / total

    signal = "neutral"
    if imbalance > 0.3:
        signal = "strong_buy_pressure"
    elif imbalance > 0.15:
        signal = "moderate_buy_pressure"
    elif imbalance < -0.3:
        signal = "strong_sell_pressure"
    elif imbalance < -0.15:
        signal = "moderate_sell_pressure"

    return {
        "imbalance": round(imbalance, 3),
        "bid_volume": round(bid_volume, 2),
        "ask_volume": round(ask_volume, 2),
        "signal": signal,
    }

```

Day 4-5 (cont): Time-Decay Exit Adjustments

python

```
# exit_manager.py additions — time-aware exits
```

```
from datetime import datetime
```

```
class TimeDecayExitManager:
```

```
    """
```

```
    Different market types need different exit timing:
```

- Sports/hourly events: tight stops, exit fast
- Weekly politics: standard stops
- Long-term markets: wider stops, more patience

```
    """
```

```
# Market type → (max_hold_hours, tighten_factor)
```

```
MARKET_PROFILES = {
```

```
    "hourly": (4, 0.5), # BTC hourly: 4h max, 50% tighter
```

```
    "daily": (24, 0.7), # Daily events: 24h max
```

```
    "sports": (12, 0.6), # Sports: 12h max
```

```
    "weekly": (120, 1.0), # Weekly: standard
```

```
    "politics": (168, 1.2), # Politics: 7 days, wider stops
```

```
    "default": (48, 1.0),
```

```
}
```

```
def classify_market(self, market: dict) -> str:
```

```
    """Classify market type from metadata."""
```

```
    question = (market.get("question", "") or "").lower()
```

```
    tags = [t.lower() for t in (market.get("tags", []) or [])]
```

```
    if any(w in question for w in ["hour", "1h", "15m"]):
```

```
        return "hourly"
```

```
    if any(w in question for w in ["today", "tonight", "game"]):
```

```
        return "daily"
```

```
    if any(t in tags for t in ["sports", "nba", "nfl", "soccer"]):
```

```
        return "sports"
```

```
    if any(t in tags for t in ["politics", "election", "president"]):
```

```
        return "politics"
```

```
    if any(w in question for w in ["this week", "by friday"]):
```

```
        return "weekly"
```

```
    return "default"
```

```
def adjust_exit_params(self, market_type: str,
```

```
    base_stop_pct: float,
```

```
    hours_held: float) -> dict:
```

```
max_hours, tighten = self.MARKET_PROFILES.get(
    market_type, self.MARKET_PROFILES["default"]
)

# Time decay: tighten stops as we approach max hold time
time_ratio = min(hours_held / max_hours, 1.0)
decay_multiplier = 1.0 - (0.5 * time_ratio) # up to 50% tighter

adjusted_stop = base_stop_pct * tighten * decay_multiplier

# Force exit if past max hold time
force_exit = hours_held >= max_hours

return {
    "adjusted_stop_pct": round(adjusted_stop, 4),
    "force_exit": force_exit,
    "hours_remaining": max(0, max_hours - hours_held),
    "market_type": market_type,
}
```

Day 6-7 (Weekend): Dashboard + Tooling

```
python
```

```
# dashboard_data.py — Generate portfolio health data
# You can display this via a simple Flask app or push to n8n
```

```
import sqlite3
```

```
import json
```

```
from collections import defaultdict
```

```
class PortfolioDashboard:
```

```
    def __init__(self, db_path="trades.db"):
```

```
        self.db_path = db_path
```

```
    def get_portfolio_summary(self) -> dict:
```

```
        conn = sqlite3.connect(self.db_path)
```

```
        # Open positions by domain
```

```
        cursor = conn.execute("""
```

```
            SELECT domain,
```

```
                   COUNT(*) as count,
```

```
                   SUM(entry_cost) as exposure,
```

```
                   SUM(current_value - entry_cost) as unrealized_pnl
```

```
            FROM positions
```

```
            WHERE status = 'open'
```

```
            GROUP BY domain
```

```
        """)
```

```
        domain_exposure = {}
```

```
        total_exposure = 0
```

```
        total_unrealized = 0
```

```
        for row in cursor:
```

```
            domain, count, exposure, unrealized = row
```

```
            domain_exposure[domain] = {
```

```
                "count": count,
```

```
                "exposure": round(exposure, 2),
```

```
                "unrealized_pnl": round(unrealized or 0, 2),
```

```
            }
```

```
            total_exposure += exposure
```

```
            total_unrealized += (unrealized or 0)
```

```
        # Concentration check
```

```
        for domain, data in domain_exposure.items():
```

```
            data["pct_of_portfolio"] = round(
```

```
                data["exposure"] / max(total_exposure, 1) * 100, 1
```

```
            )
```

```

# Recent P&L
cursor = conn.execute("""
    SELECT date(closed_at) as day,
           SUM(pnl) as daily_pnl,
           COUNT(*) as trades
    FROM trades
    WHERE status = 'closed'
    GROUP BY date(closed_at)
    ORDER BY day DESC
    LIMIT 7
""")
daily_pnl = [
    {"date": row[0], "pnl": round(row[1], 2), "trades": row[2]}
    for row in cursor
]

# Win rate
cursor = conn.execute("""
    SELECT
        COUNT(*) as total,
        SUM(CASE WHEN pnl > 0 THEN 1 ELSE 0 END) as wins
    FROM trades WHERE status = 'closed'
""")
total, wins = cursor.fetchone()
win_rate = (wins / total * 100) if total else 0

conn.close()

return {
    "total_open_positions": sum(
        d["count"] for d in domain_exposure.values()
    ),
    "total_exposure": round(total_exposure, 2),
    "total_unrealized_pnl": round(total_unrealized, 2),
    "domain_breakdown": domain_exposure,
    "daily_pnl_history": daily_pnl,
    "win_rate": round(win_rate, 1),
    "concentration_warnings": [
        f"⚠️ {d}: {data['pct_of_portfolio']}% of portfolio"
        for d, data in domain_exposure.items()
        if data["pct_of_portfolio"] > 20
    ]
}

```



```
],  
}
```

API & Tool Recommendations

Must-Have (Free / Under Budget)

Tool	Purpose	Cost	Install
<code>py-clob-client</code> v0.34.6	Polymarket trading	Free	<code>pip install py-clob-client</code>
<code>polymarket-apis</code>	Unified Poly API with Pydantic models	Free	<code>pip install polymarket-apis</code>
<code>kalshi-python-sync</code>	Kalshi trading	Free	<code>pip install kalshi-python-sync</code>
<code>pmxt</code>	Unified Poly+Kalshi+Limitless SDK	Free	<code>pip install pmxt</code>
<code>forecasting-tools</code>	Metaculus API wrapper + calibration	Free	<code>pip install forecasting-tools</code>
Gamma API	Polymarket market discovery	Free, no auth	<code>GET gamma-api.polymarket.com</code>
Metaculus API	Superforecaster probabilities	Free	<code>GET metaculus.com/api2</code>

Useful GitHub Repos (Active 2025–2026)

Repository	Stars	What It Does
<code>Polymarket/py-clob-client</code>	834	Official Python SDK
<code>pmxt-dev/pmxt</code>	—	CCXT for prediction markets
<code>Metaculus/forecasting-tools</code>	—	AI forecasting bot framework
<code>manifoldmarkets/market-maker</code>	—	Market-making bot (Manifold)
<code>CarlosIbCu/polymarket-kalshi-btc-arbitrage-bot</code>	—	BTC hourly arb between Poly+Kalshi
<code>0xalberto/polymarket-arbitrage-bot</code>	71	Intra-event Polymarket arb
<code>Drakkar-Software/OctoBot-Prediction-Market</code>	36	Open-source copy trading + arb
<code>OctagonAI/kalshi-deep-trading-bot</code>	—	AI-powered Kalshi bot

Repository	Stars	What It Does
<code>ashercn97/predmarket</code>	—	Pure-Python unified Kalshi+Poly SDK
<code>realfishsam/prediction-market-arbitrage-bot</code>	—	Cross-platform arb with pmxt

Monthly Cost Breakdown

Item	Cost
Polymarket CLOB API	Free (rate-limited)
Kalshi API	Free (rate-limited)
Metaculus API	Free
Gamma API (market data)	Free
<code>pmxt</code> SDK	Free (open source)
VPS (optional, for 24/7 scanning)	\$5–10/month
n8n (self-hosted or cloud)	Free–\$20/month
Total	\$5–\$30/month

Risk Assessment: What Could Go Wrong

Priority 1: Arbitrage

Risk	Severity	Mitigation
Execution risk: can't fill both legs simultaneously	High	Use market orders, accept slippage budget of 1–2¢
Polymarket gas fees eat profit on small arbs	Medium	Only trade arbs > 3¢ profit to cover ~1¢ gas
Market resolves ambiguously (e.g., "N/A" resolution)	Medium	Avoid markets with vague resolution criteria
Kalshi account restrictions (US-only, KYC)	Low	Verify your account status before building the integration
API rate limiting during volatile events	Medium	Implement exponential backoff, cache responses

Priority 2: Risk Management

Risk	Severity	Mitigation
Circuit breaker triggers during a winning streak	Low	Separate realized vs unrealized P&L tracking
Trailing stop whipsaws in volatile markets	Medium	Use wider trails for political markets, tighter for sports
Domain classification errors	Low	Manual override capability, periodic review

Priority 3: Entry Signals

Risk	Severity	Mitigation
Metaculus question doesn't match Polymarket question exactly	Medium	Manual verification before large bets
Order book imbalance is a lagging indicator	Medium	Combine with other signals, never trade on imbalance alone
Liquidity filter is too aggressive, misses good trades	Low	Start at \$5k volume, tune down if needed

Priority 4: Exit Optimization

Risk	Severity	Mitigation
Scale-out sells winners too early	Medium	Backtest against your 72-trade history first
Time decay forces premature exits on slow-moving markets	Low	Generous defaults, override per market type

Integration Checklist

Day 1: Risk Management

- Add domain column to positions table (run SQL)
- Add high_water_mark and remaining_fraction columns
- Classify all 182 open positions by domain
- Deploy RiskManager — wire into your main trading loop
- Deploy TrailingStopManager — replace fixed stop logic

- Test circuit breaker with a \$1 daily loss limit
- Set up n8n webhook for alerts

Day 2-3: Arbitrage Scanner

- Deploy PolymarketArbitrageScanner
- Run first scan, log results to SQLite
- Set up 5-minute cron job or loop
- Wire alerts to n8n → Telegram/Slack
- (Optional) Set up Kalshi account + API key
- (Optional) Deploy BTC hourly cross-platform scanner

Day 4-5: Entry Filters

- Add LiquidityFilter before all entries
- Test MetaculusComparison on 10 current markets
- Add OrderBookImbalanceDetector as supplementary signal
- Deploy TimeDecayExitManager

Day 6-7: Dashboard + Review

- Deploy PortfolioDashboard
- Generate first portfolio health snapshot
- Review concentration warnings
- Backtest trailing stops against your 72 closed trades
- Document learnings, tune parameters